

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..41

HEALTH CHECKS

 Copy as Markdown

Health checks

When your server runs behind a host that can restart it or route traffic away from it, that host needs a way to ask how the server is doing. Serverpod answers three such questions over HTTP, at URLs that match what container platforms like Kubernetes expect.

This page covers two separate things that share the word "health":

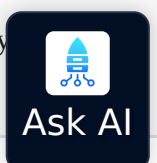
- **Health probes**, the HTTP endpoints something else calls to decide whether to send you traffic or restart you.
- **Health metrics**, numbers Serverpod records about itself into your database for you to look at later.

Health probes

The three endpoints are always available, with no setup. Every server your Serverpod instance runs answers them, so in a default development configuration they respond on the API server's port `8080`, the Insights server's `8081`, and the web server's `8082` when your project has one. Point your host at whichever port it can reach.

These three paths are reserved. A [web server route](#) registered on them would never be reached.

Endpoint	Question it answers	Returns
<code>/livez</code>	Is the process still working, or should it be restarted?	<code>200</code> if the server can respond at all.
<code>/readyz</code>	Should traffic be sent here right now?	<code>200</code> when dependencies are healthy when they are not.



Endpoint	Question it answers	Returns
<code>/startup</code> <code>pz</code>	Has the server finished starting?	<code>200</code> once startup is complete.

```
curl http://localhost:8080/readyz
```

Liveness is deliberately permissive. It only reports failure when the process is broken beyond recovery, because failing it means a restart. A slow database or a brief network problem should not restart your server, so `/livez` does not check dependencies at all.

Readiness is the one that controls traffic. It checks the database, Redis when it is enabled, and any custom indicators you add. Failing readiness stops new traffic without restarting the process, which gives a struggling server room to recover.

Startup exists so the other two are not consulted too early. While it is failing, a platform holds off its liveness and readiness probes. In practice Serverpod opens its HTTP listeners as the last step of starting, so a probe sent during startup gets a refused connection rather than a `503`.

Response format

The probes follow the [draft standard for health check responses](#).

Requests without valid authentication get the status code and an empty body, so nothing about your dependencies is exposed publicly. An authenticated request gets the same status code plus a body. Any credential your server's authentication handler accepts unlocks the body, with no particular scope required.

```
{
  "status": "pass",
  "time": "2026-01-14T10:30:00Z",
  "checks": {
    "database:connection": [
      {
        "componentType": "datastore",
        "status": "pass",
        "observedValue": 12,
        "observedUnit": "ms",
        "time": "2026-01-14T10:30:00Z"
      }
    ],
    "redis:connection": [
      {
        "componentType": "datastore",
        "status": "pass",
        "time": "2026-01-14T10:30:00Z"
      }
    ]
  }
}
```

```
]
}
}
```

When a check fails, the response also carries `notes` listing which ones. The `checks` object is left out when there is nothing to report, which is the normal case for `/livez`.

Built-in indicators

Serverpod registers these based on your configuration:

- `serverpod:startup` records that the server has begun starting.
- `database:connection` checks the database, when one is configured.
- `redis:connection` checks Redis, when it is [enabled](#).

Add your own indicator

Extend `HealthIndicator` to check something your server depends on, such as an external API:

```
import 'package:serverpod/serverpod.dart';

class StripeApiIndicator extends HealthIndicator<double> {
  @override
  String get name => 'stripe:api';

  @override
  String get componentType => HealthComponentType.component.name;

  @override
  String get observedUnit => 'ms';

  @override
  Duration get timeout => const Duration(seconds: 3);

  @override
  Future<HealthCheckResult> check() async {
    final stopwatch = Stopwatch()..start();
    try {
      await stripeClient.ping();
      stopwatch.stop();
      return pass(observedValue: stopwatch.elapsedMilliseconds.toDouble());
    } catch (e) {
      return fail(output: 'Stripe API unavailable: $e');
    }
  }
}
```

The type parameter is the type of `observedValue`, which is what the check reports alongside pass or fail. Use `output` to attach a message, as the failure branch above does. Override `componentId` when several

instances of the same component exist, such as `primary-db` and `replica-db`, and the response should say which one answered. The built-in indicators leave it unset.

Register it through `healthConfig` when you create the server, choosing the list by which probe should run it:

```
final pod = Serverpod(
  args,
  healthConfig: HealthConfig(
    cacheTtl: Duration(seconds: 2),
    additionalReadinessIndicators: [StripeApiIndicator()],
    additionalStartupIndicators: [CacheWarmupIndicator()],
  ),
);
```

- `additionalReadinessIndicators` are checked by `/readyz`, so use them for dependencies that must be available to serve traffic.
- `additionalStartupIndicators` are checked by `/startupz`, so use them for work that has to finish before the server is ready at all, such as warming a cache.

The `cacheTtl` option sets how long a result is reused before the check runs again, which keeps frequent probing from hammering your dependencies. It defaults to one second. Each indicator can set its own `timeout`, five seconds by default, so one slow check cannot hold up the whole response.

Health metrics

Separately from the probes, Serverpod records numbers about itself: CPU, memory, and how long the database takes to respond. These go into the `serverpod_health_metric` and `serverpod_health_connection_info` tables, and Insights charts them.

Collection runs once per `healthCheckInterval`, one minute by default. Setting the interval to zero turns collection off. A few conditions apply: a cycle writes nothing when the database has not been used since the last one, collection only runs in the `monolith` and `maintenance` [server roles](#), and it does not run on Windows.

Older rows are folded up rather than kept forever: minute rows become hourly after two days, and hourly rows become daily after about a month.

Record your own metric

A `HealthCheckHandler` runs on the same schedule as the built-in metrics and returns whatever you want recorded:

```
Future<List<ServerHealthMetric>> myHealthCheckHandler(
```

```

Serverpod pod,
DateTime timestamp,
) async {
  return [
    ServerHealthMetric(
      name: 'MyMetric',
      serverId: pod.serverId,
      timestamp: timestamp,
      isHealthy: true,
      value: 1.0,
      granularity: 1,
    ),
  ];
}

```

The `granularity` field is the period the value covers, in minutes. Use `1` for values recorded on the normal cycle, since Serverpod produces the hourly and daily rows itself.

Register the handler on the server:

```

final pod = Serverpod(
  args,
  healthCheckHandler: myHealthCheckHandler,
);

```

NOTE

The two APIs read alike but are not related. Use `healthConfig` with `HealthIndicator` for the HTTP probes, and `healthCheckHandler` with `ServerHealthMetric` for the recorded metrics.

Related

- [Logging](#): the other half of knowing what your server is doing.
- [Insights](#): charts for the collected metrics.
- [Load testing](#): reading these probes and metrics while generating concurrent traffic.
- [Custom hosting](#): wiring the probes up to your host.